



FareLens

Airline fare rules, explained.

Self-hosted API + admin UI that turns raw airline fare-rules text into traveller-friendly summaries and answers questions about them — using the LLM provider you choose.

Product & Technical Guide · Version 1.0.0

September 2026

Built by TravelSwitch for the travel-tech community · travelswitch.com
Open source under the MIT License

Contents

1. Introduction	3
1.1 Why FareLens	3
1.2 What you get	3
1.3 The admin interface at a glance	4
2. Quick start.....	5
2.1 Run it.....	5
2.2 Choose an LLM provider	5
2.3 Create an API key	5
2.4 Call the API.....	6
2.5 Try it in the Playground	6
2.6 Watch usage	8
3. API reference	9
3.1 Authentication and errors	9
3.2 POST /api/v1/fare-rules/summary	9
3.3 POST /api/v1/fare-rules/chat/stream	10
3.4 Other endpoints.....	10
3.5 Interactive documentation (Swagger UI).....	10
4. Configuration	13
4.1 Environment variables	13
4.2 LLM providers	13
4.3 Using your own Postgres / Redis	14
4.4 Prompts	14
5. Architecture	16
5.1 Request flow	16
5.2 Code layout.....	16
5.3 Technology.....	16
6. Deployment	17
6.1 Production checklist	17
6.2 Reverse proxy	17
6.3 Scaling, persistence and upgrades.....	17
6.4 What FareLens stores	18
7. Contributing, credits and license	19

1. Introduction

1.1 Why FareLens

Airline fare rules are long, uppercase, telegraphic and full of ATPCO jargon (**NON-REF**, **RFND**, **CHG PEN**, **NOSHOW**...). Travellers just want to know: **what does it cost me to cancel or change?** FareLens answers that in plain language, in the traveller's own language, formatted for desktop or mobile — and caches the result so repeated fare rules cost zero tokens.

FareLens was built by TravelSwitch, where it powers fare-rule explanations in production booking flows, and is shared with the travel-tech community so any airline, OTA or travel-tech team can run it on their own stack.

1.2 What you get

Capability	Description
Summary API	<code>POST /api/v1/fare-rules/summary</code> → Markdown "mini rules": a table on desktop, compact sections on mobile.
Chat API	<code>POST /api/v1/fare-rules/chat/stream</code> → Server-Sent Events Q&A over one or many itinerary segments; segment- and date-aware.
Bring your own model	OpenAI · Azure AI (Azure OpenAI) · Anthropic Claude · Google Gemini · Groq · AWS Bedrock. Pick one in the UI, test it, switch any time.
Multilingual	Arabic, Urdu, French, German, Hindi, Chinese and more — the response language is a request parameter.
Caching	Content-digest cache: Redis → Postgres → model. Identical rules never hit the LLM twice.
Admin UI	Provider setup, API keys, usage dashboard, datastore management, playground, account.
One command	<code>docker compose up -d</code> brings up the API, Postgres and Redis with zero configuration.

1.3 The admin interface at a glance

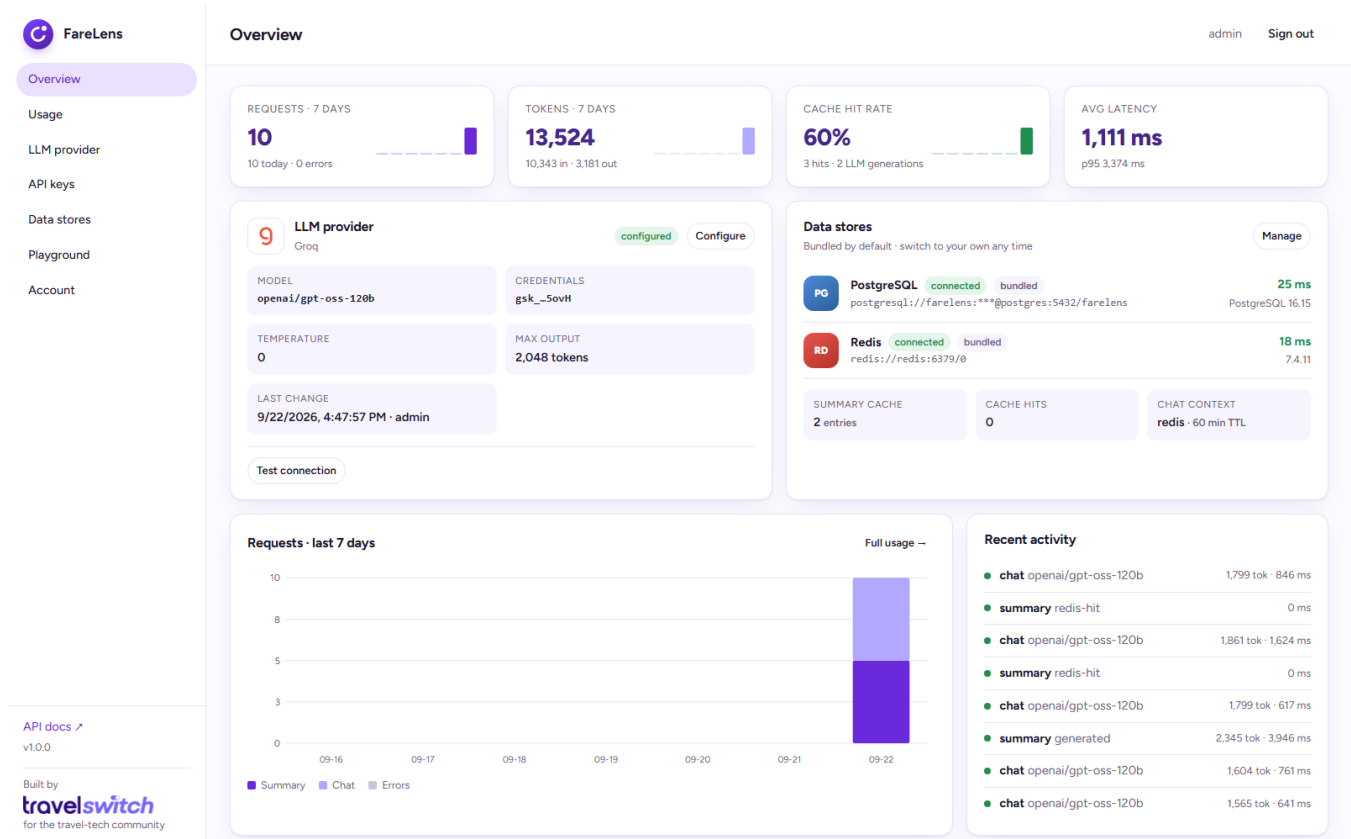


Figure 1 – Overview dashboard: usage tiles, provider status, live datastore checks, recent activity

2. Quick start

Requirements: Docker with Compose v2. Nothing else — no Python, no database to install.

2.1 Run it

```
git clone https://github.com/travelswitch/farelens.git
cd farelens
docker compose up -d
```

Open **http://localhost:8000** and sign in with the default credentials **admin** / **admin**. The UI asks you to change the password on first login.

2.2 Choose an LLM provider

Pick a provider, paste your key, click **Test connection**, then **Save & activate**. Credentials are encrypted at rest. Any model id the provider accepts can be typed in; the suggested list is only a starting point.

The screenshot displays the FareLens web interface for configuring an LLM provider. On the left, a sidebar contains navigation links: Overview, Usage, LLM provider (highlighted), API keys, Data stores, Playground, and Account. The main panel is titled 'LLM provider' and features a 'Choose an inference provider' section with buttons for OpenAI, Azure AI (Azure OpenAI), Anthropic (Claude), Google (Gemini), Groq (marked as 'active'), and AWS Bedrock. Below this, the 'Active configuration' for Groq is detailed, showing the model 'openai/gpt-oss-120b', API key 'gsk_5ovHt', temperature '0', and max output tokens '2048'. Action buttons include 'Test connection', 'Save & activate', and 'Remove'. A 'Good to know' section provides additional context on stored secrets and cached summaries. The bottom left corner includes links to 'API docs v1.0.0' and 'Built by travelswitch for the travel-tech community'.

Figure 2 – Provider picker with connection test

2.3 Create an API key

API keys authenticate the public endpoints. The full key is shown once; only a hash is stored.

API keys

Create a key

Name, e.g. booking-web Create API key

Send it as X-API-Key: f1_ (or Authorization: Bearer f1_). The full key is shown once.

Keys

NAME	PREFIX	CREATED	LAST USED	REQUESTS	STATUS
test	f1_JlUfKt5fR...	9/22/2026, 4:46:53 PM admin	9/22/2026, 4:48:50 PM	4	active Revoke

API docs v1.0.0

Built by **travelswitch** for the travel-tech community

Figure 3 – API key management

2.4 Call the API

```
curl -X POST http://localhost:8000/api/v1/fare-rules/summary \
-H "X-API-Key: f1_..." \
-H "Content-Type: application/json" \
-d '{"fare_rules_text": "CANCELLATIONS BEFORE DEPARTURE CHARGE SAR 150 FOR CANCEL/REFUND ...",
"lang": "en"}'
```

Interactive OpenAPI documentation is served at **http://localhost:8000/docs** (see Section 3.5).

2.5 Try it in the Playground

Both endpoints can be exercised from the UI without an API key (the admin session is used). Load a sample, choose a language and layout, and see the result rendered.



Overview
Usage
LLM provider
API keys
Data stores
Playground
Account

Playground

admin Sign out

Summary Chat

Runs with your admin session - recorded in Usage

Fare rules

Paste raw airline text — HTML and uppercase are fine

Load sample...

CANCELLATIONS
BEFORE DEPARTURE
CHARGE SAR 150 FOR CANCEL/REFUND.
CHILD/INFANT DISCOUNTS APPLY.
WAIVED FOR DEATH OF PASSENGER OR FAMILY MEMBER.
AFTER DEPARTURE
TICKET IS NON-REFUNDABLE.
REFUND OF UNUSED TAXES PERMITTED.
NO-SHOW
TICKET IS NON-REFUNDABLE.

CHANGES
BEFORE DEPARTURE
CHARGE SAR 100 FOR REISSUE/REVALIDATION. FARE DIFFERENCE APPLIES.
CHANGES PERMITTED UP TO 4 HOURS BEFORE DEPARTURE.
AFTER DEPARTURE
CHARGE SAR 200 FOR REISSUE. FARE DIFFERENCE APPLIES.
NO-SHOW
CHARGE SAR 250 FOR REISSUE. FARE DIFFERENCE APPLIES.

Language English Layout Desktop Mobile Bypass cache 589 chars

Summarise →

Result

cache · redis groq · openai/gpt-oss-120b · 3 ms

Copy Markdown Raw

Section	Condition / Timing	Penalty	Applicability	Notes
Cancellation / Refund	Before departure	SAR 150	Per passenger	Child/infant discounts apply; waived for death of passenger or family member
Cancellation / Refund	After departure	Non-refundable	Per passenger	Refund of unused taxes permitted
Cancellation / Refund	No-show	Non-refundable	Per passenger	—
Change / Reissue	Before departure (up to 4 h before departure)	SAR 100	Per passenger	Fare difference applies; changes permitted up to 4 h before departure
Change / Reissue	After departure	SAR 200	Per passenger	Fare difference applies
Change / Reissue	No-show	SAR 250	Per passenger	Fare difference applies
Other	Ticket validity	Not applicable	All	Ticket valid for 1 year from date of issue

Key points

- No refund is available after departure or for a no-show (taxes only refundable after departure).
- Changes can be made up to 4 hours before departure for SAR100 plus any fare difference.
- Ticket remains valid for 1 year from the issue date.

API docs
v1.0.0

Built by
travelswitch
for the travel-tech community

Figure 4 – Playground: fare-rules summary with cache badge, model and latency



Overview
Usage
LLM provider
API keys
Data stores
Playground
Account

Playground

admin Sign out

Summary Chat

Runs with your admin session - recorded in Usage

Itinerary

One segment per flight, each with its own fare rules

+ Segment

DEL → DXB 29-Sep-2026

Fare rules for this segment - 589 chars

CANCELLATIONS
BEFORE DEPARTURE
CHARGE SAR 150 FOR CANCEL/REFUND.
CHILD/INFANT DISCOUNTS APPLY.
WAIVED FOR DEATH OF PASSENGER OR FAMILY MEMBER.
AFTER DEPARTURE
TICKET IS NON-REFUNDABLE.
REFUND OF UNUSED TAXES PERMITTED.

DXB → DEL 06-Oct-2026

Fare rules for this segment - 589 chars

CANCELLATIONS
BEFORE DEPARTURE
CHARGE SAR 300 FOR CANCEL/REFUND.
CHILD/INFANT DISCOUNTS APPLY.
WAIVED FOR DEATH OF PASSENGER OR FAMILY MEMBER.
AFTER DEPARTURE
TICKET IS NON-REFUNDABLE.
REFUND OF UNUSED TAXES PERMITTED.

Language English Layout Desktop Mobile

Conversation

Conversation 1e8561aa... · openai/gpt-oss-120b · 1,836 tokens · 6,197 ms

Reset

What does it cost to cancel the outbound flight? You



Outbound flight (DEL → DXB) – cancellation before departure: SAR150 (child/infant discounts apply; waived for death of passenger or family member).

Segment	Operation	Timing	Penalty
DEL-DXB	Cancel/Refund	Before departure	SAR150 (discounts for child/infant; waiver for death)

After departure the ticket is non-refundable (only unused taxes may be refunded).

e.g. Can I cancel the return flight on 2026-10-01?

Send

API docs
v1.0.0

Built by
travelswitch
for the travel-tech community

Figure 5 – Playground: multi-segment chat with streaming answers

2.6 Watch usage

Every request is recorded: tokens in/out, latency, cache status, language and API key. The Usage page aggregates per day, per feature, per model and per key.

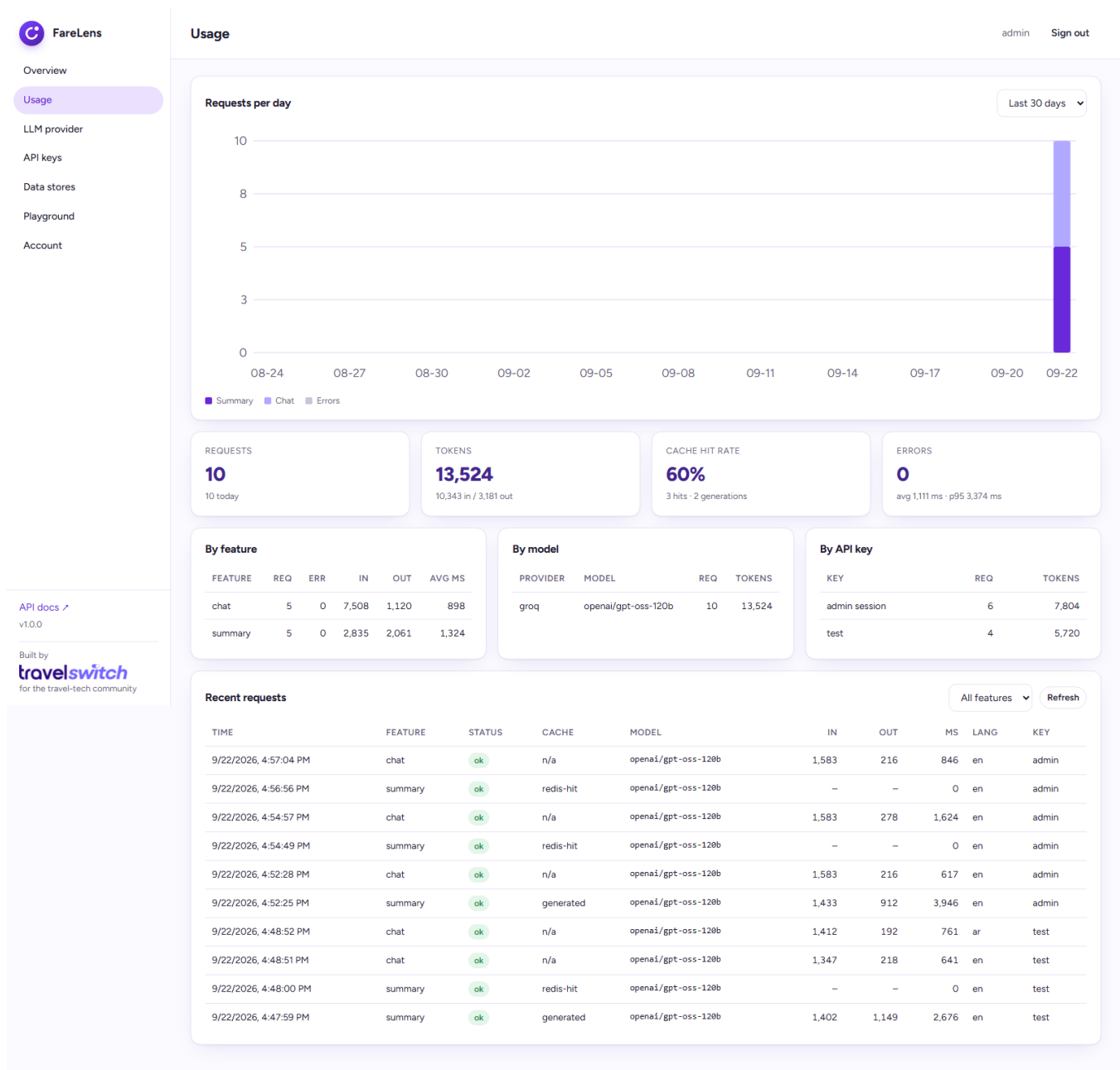


Figure 6 – Usage analytics

3. API reference

3.1 Authentication and errors

All public endpoints require an API key in the `X-API-Key` header (or `Authorization: Bearer fl_...`). Requests are rate-limited per key (default 120 per 60 s). Every response carries an `X-Request-Id` header; errors share one JSON envelope:

```
{"request_id": "...", "error": {"code": "LLM_NOT_CONFIGURED", "message": "...", "http_status": 503}}
```

HTTP	Code	Meaning
401	UNAUTHORIZED / INVALID_API_KEY	Missing or revoked key.
413	PAYLOAD_TOO_LARGE	Fare-rules text exceeds <code>MAX_FARE_RULES_CHARS</code> .
422	VALIDATION_ERROR / CONVERSATION_EXPIRED / SEGMENTS_REQUIRED	Bad request shape, or the chat context is gone — resend segments.
429	TOO_MANY_REQUESTS	Rate limit; see <code>Retry-After</code> .
502 / 503 / 504	LLM_*	Upstream provider problem: auth, model not found, rate limited, timeout, or no provider configured.

3.2 POST /api/v1/fare-rules/summary

Field	Type	Notes
<code>fare_rules_text</code>	string	Raw fare rules; HTML is stripped. Required.
<code>lang</code>	string	Response language code, default <code>en</code> .
<code>is_mobile_view</code>	bool	Compact sections instead of tables.
<code>bypass_cache</code>	bool	Force regeneration.

Response:

```
{
  "summary_markdown": "| Section | Condition / Timing | Penalty | Applicability | Notes |\n...",
  "cache": "generated",           // redis-hit | postgres-hit | generated (also in X-Cache
header)
  "provider": "groq",
  "model": "openai/gpt-oss-120b",
  "lang": "en",
  "is_mobile_view": false,
  "latency_ms": 1840
}
```

3.3 POST /api/v1/fare-rules/chat/stream

Start a conversation by sending the itinerary's segments. Follow-ups only need the `convo_id`, the new `user_message` and the client-side `history`. Segments are retained for `CONVERSATION_TTL_SECONDS` (default 1 hour).

```
{
  "segments": [
    {"source_airport": "DEL", "destination_airport": "DXB", "departure_date": "2026-11-06",
    "fare_rules_text": "..."},
    {"source_airport": "DXB", "destination_airport": "DEL", "departure_date": "2026-11-12",
    "fare_rules_text": "..."}
  ],
  "user_message": "Can I change the return flight?",
  "history": [],
  "lang": "en",
  "is_mobile_view": false
}
```

The response is `text/event-stream`:

```
event: start    data: {"convo_id": "...", "segments": [...]}
event: token    data: {"content": "Yes – "}
event: token    data: {"content": "changes on the return ..."}
event: end      data: {"convo_id": "...", "usage": {"prompt_tokens": 1450, "completion_tokens": 92},
"latency_ms": 2100}
event: error    data: {"error": {"code": "...", "message": "..."}}
```

Follow-up message:

```
{"convo_id": "...", "user_message": "And if I no-show?", "history": [{"role": "user", "content":
"..."}, {"role": "assistant", "content": "..."}]}
```

3.4 Other endpoints

Endpoint	Purpose
POST /api/v1/fare-rules/chat	Same request as the stream endpoint; single JSON response.
DELETE /api/v1/fare-rules/chat/{convo_id}	Forget a conversation's stored segments early.
GET /health	Liveness probe (reports Postgres/Redis connectivity).
GET /health/ready	Readiness probe (503 until Postgres is reachable; reports whether an LLM is configured).

3.5 Interactive documentation (Swagger UI)

The OpenAPI 3.1 specification is served at `/openapi.json` and rendered at `/docs` (Swagger UI) and `/redoc`. Every request and response schema can be inspected and tried live.

FareLens

1.0.0 OAS 3.1

[/openapi.json](#)

FareLens turns raw airline fare rules into traveller-friendly summaries and answers questions about them, using the LLM provider you configure (OpenAI, Azure OpenAI, Anthropic, Google Gemini, Groq or AWS Bedrock).

Authenticate public endpoints with an API key created in the admin UI:

X-API-Key: fl_...

health

GET /health Liveness probe

GET /health/ready Readiness probe

fare-rules

POST /api/v1/fare-rules/summary Summarise fare rules into traveller-friendly Markdown

POST /api/v1/fare-rules/chat/stream Ask a question about the fare rules (Server-Sent Events)

POST /api/v1/fare-rules/chat Ask a question about the fare rules (single JSON response)

DELETE /api/v1/fare-rules/chat/{convo_id} Forget a conversation's stored segments

Schemas

ChatMessage > Expand all object

ChatRequest > Expand all object

ChatResponse > Expand all object

ConversationDeleteResponse > Expand all object

HTTPValidationError > Expand all object

JourneySegment > Expand all object

SummaryRequest > Expand all object

SummaryResponse > Expand all object

ValidationError > Expand all object

Figure 7 – Swagger UI at /docs

FareLens ^{1.0.0} ^{OAS 3.1}

[OpenAPI JSON](#)

FareLens turns raw airline fare rules into traveller-friendly summaries and answers questions about them, using the LLM provider you configure (OpenAI, Azure OpenAI, Anthropic, Google Gemini, Groq or AWS Bedrock). Authenticate public endpoints with an API key created in the admin UI.

X-API-Key: fl_...

health

GET /health Liveness probe

GET /health/ready Readiness probe

fare-rules

POST /api/v1/fare-rules/summary Summarise fare rules into traveller-friendly Markdown

Turns raw airline fare rules text into a concise Markdown summary covering cancellation/refund, change/reissue and no-show conditions. Results are cached (Redis → Postgres) by content digest; the X-Cache response header reports `redis-hit`, `postgres-hit` or `generated`.

Parameters

Try it out

No parameters

Request body

required

application/json

Example Value Schema

```
{
  "fare_rules_text": "string",
  "lang": "en",
  "is_mobile_view": false,
  "bypass_cache": false
}
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls accept header

Example Value Schema

```
{
  "summary_markdown": "string",
  "cache": "redis-hit",
  "provider": "string",
  "model": "string",
  "lang": "string",
  "is_mobile_view": true,
  "bypass_cache": false
}
```

422 Validation Error

No links

Media type

application/json

Example Value Schema

```
{
  "detail": {
    "loc": [
      {
        "msg": "string",
        "type": "string",
        "input": "string",
        "ctx": {}
      }
    ]
  }
}
```

POST /api/v1/fare-rules/chat/stream Ask a question about the fare rules (Server-Sent Events)

POST /api/v1/fare-rules/chat Ask a question about the fare rules (single JSON response)

DELETE /api/v1/fare-rules/chat/{conv_id} Forget a conversation's stored segments

Schemas

ChatMessage > Expand all object

ChatRequest > Expand all object

ChatResponse > Expand all object

ConversationDeleteResponse > Expand all object

HTTPValidationError > Expand all object

JourneySegment > Expand all object

SummaryRequest > Expand all object

SummaryResponse > Expand all object

ValidationError > Expand all object

Figure 8 – Summary endpoint expanded: request schema and responses

4. Configuration

FareLens works with zero configuration. Copy `.env.example` to `.env` to change anything.

4.1 Environment variables

Variable	Default	Purpose
<code>APP_SECRET_KEY</code>	(generated)	Signs sessions and encrypts stored credentials. Set it in production. If empty, one is generated into <code>APP_DATA_DIR/secret.key</code> .
<code>APP_DATA_DIR</code>	<code>/data</code> (Docker)	Local state: generated secret, datastore overrides.
<code>DATABASE_URL</code> / <code>REDIS_URL</code>	bundled containers	Default datastores.
<code>ADMIN_DEFAULT_USERNAME</code> / <code>ADMIN_DEFAULT_PASSWORD</code>	<code>admin</code> / <code>admin</code>	Seeded once, on first start with an empty database.
<code>MAX_FARE_RULES_CHARS</code>	<code>60000</code>	Max fare-rules length per request / segment.
<code>SUMMARY_CACHE_NAMESPACE</code>	<code>v1</code>	Bump to invalidate all cached summaries (e.g. after editing prompts).
<code>SUMMARY_CACHE_TTL_SECONDS</code>	<code>2592000</code>	Redis TTL for summaries; Postgres keeps them indefinitely.
<code>CONVERSATION_TTL_SECONDS</code>	<code>3600</code>	Retention of chat segments per <code>convo_id</code> .
<code>RATE_LIMIT_REQUESTS</code> / <code>RATE_LIMIT_WINDOW_SECONDS</code>	<code>120</code> / <code>60</code>	Sliding window per API key.
<code>CORS_ALLOW_ORIGINS</code>	<code>*</code>	Comma-separated origins for browser clients.
<code>LLM_TIMEOUT_SECONDS</code>	<code>90</code>	Upstream request timeout.

4.2 LLM providers

Provider	Credentials	Notes
OpenAI	API key, optional base URL	Base URL makes it work with any OpenAI-compatible gateway.
Azure AI (Azure OpenAI)	Endpoint, API key, API version	Model = your deployment name .
Anthropic (Claude)	API key	Sampling parameters are omitted (current Claude models reject them).
Google (Gemini)	API key	Google AI Studio keys.
Groq	API key	Fast open-weight models; Groq retires models regularly.
AWS Bedrock	Region, optional access key/secret	Empty keys → container IAM role. Uses the Converse API, so any Converse-capable model works.

Credentials are Fernet-encrypted with `APP_SECRET_KEY`. The public API never sees them; the admin UI shows them masked and can reveal them to a logged-in admin.

4.3 Using your own Postgres / Redis

The **Data stores** page lets you point FareLens at an existing Postgres and/or Redis with plain host / port / user / password fields (or a pasted connection string). It tests the connection, creates the schema, copies users, LLM configuration and API keys if the target is empty, and switches live — no restart. The choice is persisted, encrypted, in the data volume. Redis is optional: without it, summaries are cached in Postgres only and chat context lives in memory (single instance).

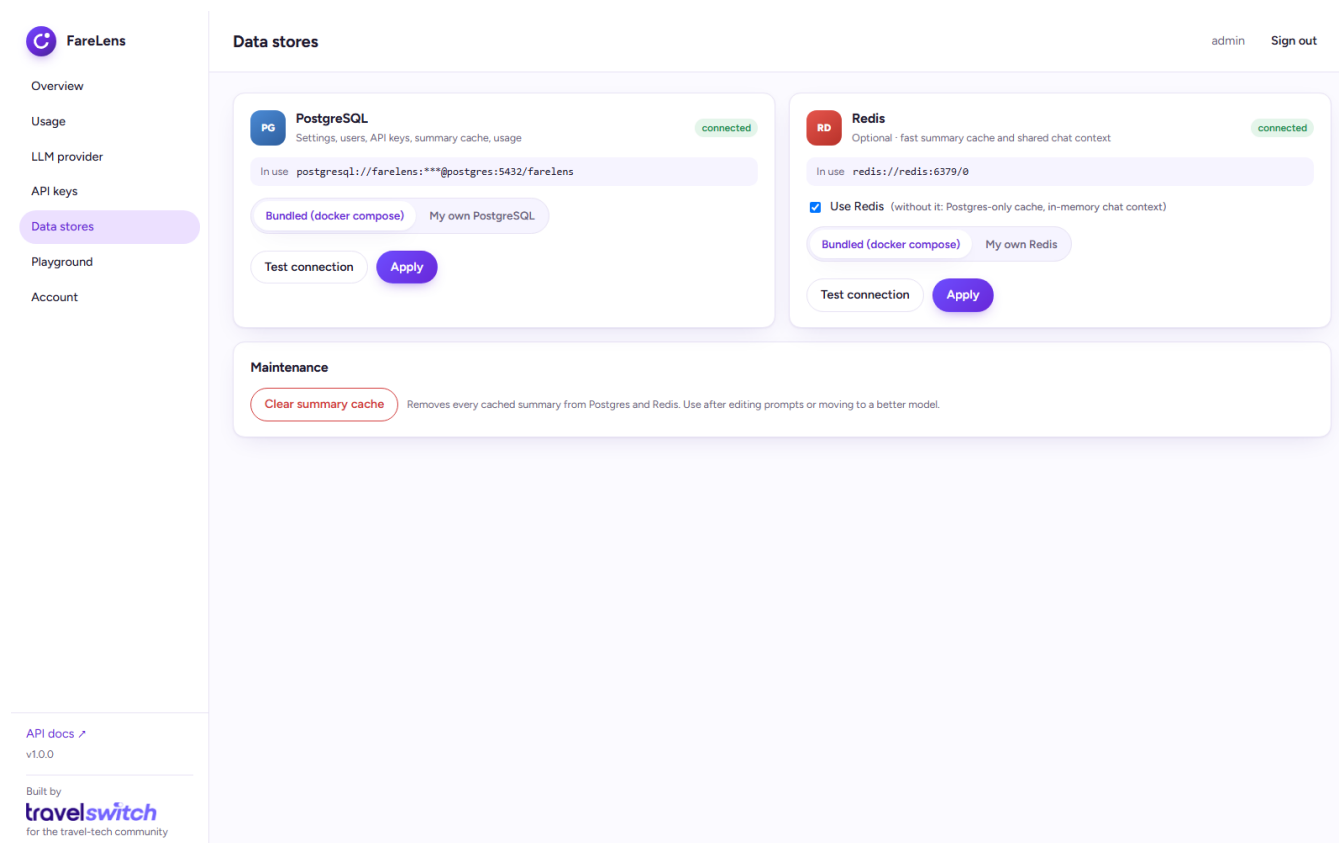


Figure 9 – Data stores: bundled or your own, with connection tests

4.4 Prompts

Prompts can be edited from the admin UI (**Prompts** page) or on disk. The UI validates required placeholders, previews the edited prompt against sample fare rules before saving, stores overrides in Postgres (so they apply to every instance and survive upgrades) and offers a one-click reset to the file default. Saving a summary prompt clears the summary cache so the change takes effect immediately.

[Overview](#)
[Usage](#)
[LLM provider](#)
[API keys](#)
[Data stores](#)
[Prompts](#)
[Playground](#)
[Account](#)

API docs

v1.0.0

Built by

for the travel-tech community

admin

Sign out

Summary · system prompt

default

Role, extraction rules and accuracy policy for the fare-rules summary.

[[LANG]]

[[LANG_NAME]]

[[LAYOUT_GUIDELINES]]

You are FareLens, an airline fare-rules specialist. You convert raw fare rules (ATPCO / GDS category text, airline "penalties" pages, NDC fare-rule blobs) into a short, accurate summary a traveller or travel agent can act on.

Output language

Write the ENTIRE response in [[LANG_NAME]] (code: [[LANG]]), using only the native script of that language. Do not mix scripts (no Latin, Chinese, or other scripts inside Arabic/Urdu text, and vice versa). Keep IATA airport codes, airline codes, currency codes (USD, SAR, INR...), fare-basis codes and numbers exactly as they appear. If [[LANG_NAME]] is written right-to-left the container already sets the direction – do not add HTML or direction markers.

What to extract

Cover each of these areas when the rules mention them; skip an area entirely only if the text contains nothing about it:

1. ****Cancellation / Refund**** – before departure, after departure (partially used ticket), and no-show. Include the penalty per condition, whether the fare is non-refundable, and whether unused taxes/fees are refundable when stated.
2. ****Change / Reissue (date or flight change)**** – before departure, after departure, and no-show. Include the change fee AND note that fare difference applies when stated. Mention if the ticket must be reissued before original departure, if the new travel must be on the same airline, or if changes are only permitted before ticketing.
3. ****No-show**** – the explicit no-show rule, or the rule that applies when the passenger does not cancel before departure.
4. ****Other conditions that affect money or eligibility**** – e.g. ticket validity, minimum/maximum stay, advance-purchase requirements, combinability limits, penalties applied per passenger / per direction / per ticket, child or infant discounts on penalties, penalties that vary by booking class, fees "waived" for specific reasons (death, illness, involuntary schedule

3,899 chars

Preview language

English

☐ Mobile layout

☒ Clear summary cache on save

Preview with sample

Save

Show default

Figure 8 – Prompt editor with live preview

On disk, prompts live in `prompts/fare_rules/*.md` and are hot-reloaded on change. `summary_system.md` and `chat_system.md` carry the domain rules — ATPCO vocabulary, before/after-departure and no-show logic, a strict no-hallucination policy — while the `*_layout_*.md` files control desktop versus mobile formatting. After changing a prompt, bump `SUMMARY_CACHE_NAMESPACE` or clear the cache from the UI.

Built by TravelSwitch for the travel-tech community

Page 15 of 19

5. Architecture

5.1 Request flow

1. The client calls the API with an `X-API-Key`; the key hash is verified (cached for 60 s) and the request is rate-limited.
2. Fare-rules text is normalised (HTML stripped, whitespace and case canonicalised) and hashed with SHA-256 together with the language, layout and cache namespace.
3. The digest is looked up in Redis, then Postgres. A hit returns immediately (`X-Cache: redis-hit / postgres-hit`).
4. On a miss, the prompt is rendered from the templates and sent to the configured provider adapter.
5. The result is stored in Postgres and Redis, a usage event is recorded (tokens, latency, cache status, key), and the response is returned.

Chat follows the same provider path but streams tokens over Server-Sent Events; segment context is stored per conversation in Redis (or memory) with a TTL.

5.2 Code layout

```
farelens/
  api/          FastAPI routes (public v1 + admin), auth dependencies, middleware
  services/     summary (cache → LLM), chat (SSE), usage, API keys, users, LLM config
  llm/          provider registry + adapters (openai/azure/groq, anthropic, google, bedrock)
  db/           asyncpg / redis managers, live datastore switching, SQL migrations
  core/         settings, security (bcrypt · JWT · Fernet), error envelope
  prompts/      editable prompt templates
  migrations/   forward-only SQL applied at startup
  ui/           admin SPA (vanilla JS, no build step)
docker-compose.yml · Dockerfile · tests/ · .github/workflows/ci.yml
```

5.3 Technology

Layer	Choice
Runtime	Python 3.12, FastAPI, Uvicorn (async end-to-end)
Datastores	PostgreSQL 16 (asyncpg), Redis 7 (optional)
LLM SDKs	openai (OpenAI, Azure, Groq), anthropic, google-genai, boto3 (Bedrock Converse)
Security	bcrypt passwords, HMAC-signed session cookies with CSRF guard, Fernet-encrypted secrets, SHA-256 API-key hashes
Packaging	Multi-stage Docker image, non-root user, healthchecks; GitHub Actions CI (ruff, pytest, image build)

6. Deployment

6.1 Production checklist

- ☐ Set `APP_SECRET_KEY` to a long random value and never change it afterwards (it encrypts stored credentials).
- ☐ Change the admin password (Account → Change password).
- ☐ Change the bundled Postgres password, or use your own Postgres.
- ☐ Serve over HTTPS behind a reverse proxy; the session cookie becomes `Secure` automatically.
- ☐ Restrict `CORS_ALLOW_ORIGINS` if browsers call the API directly.
- ☐ Keep Postgres and Redis on the private network — the compose file does not publish their ports.

6.2 Reverse proxy

FareLens listens on plain HTTP (port 8000) and honours `X-Forwarded-*` headers. Server-Sent Events require response buffering to be **off** on every proxy in the path.

```
# nginx
location / {
    proxy_pass http://127.0.0.1:8000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_buffering off;      # required for SSE
    proxy_read_timeout 300s;
}
```

6.3 Scaling, persistence and upgrades

- Run several `app` replicas behind the proxy with Redis enabled; provider-config changes propagate within 30 s. Without Redis, keep a single replica.
- Back up the Postgres volume (`pg_dump`) and the `/data` volume (it holds the generated secret when `APP_SECRET_KEY` is not set).
- Upgrade with `git pull && docker compose build app && docker compose up -d`; schema migrations run automatically and are safe to run from multiple replicas.
- Usage rows are small but unbounded — prune `usage_events` periodically if traffic is heavy.

6.4 What FareLens stores

Data	Where	Protection
LLM API keys / cloud credentials	Postgres <code>llm_config.secrets</code>	Fernet-encrypted with <code>APP_SECRET_KEY</code>
External datastore DSNs	<code>APP_DATA_DIR/datastores.json</code>	Fernet-encrypted, file mode 600
Admin passwords	Postgres <code>users</code>	bcrypt (12 rounds)
API keys	Postgres <code>api_keys</code>	SHA-256 hash + display prefix
Fare-rules summaries	Redis + Postgres cache	Plain text (no PII expected)
Chat context (segments)	Redis (TTL) or memory	Plain text, expires
Usage events	Postgres	Metadata only — no request bodies

7. Contributing, credits and license

Issues and pull requests are welcome. Run `ruff check farelens tests` and `pytest -q` before opening a PR; see [CONTRIBUTING.md](#) for the full guide, including how to add a new LLM provider (a registry entry plus an adapter — the UI form is generated automatically).

Security issues should be reported privately as described in [SECURITY.md](#).

Provider icons in the admin UI are from lobe-icons (MIT). Brand names and logos remain the property of their respective owners.

FareLens is released under the MIT License. © 2026 TravelSwitch.

Built by TravelSwitch for the travel-tech community · <https://travelswitch.com>